

SOFTWARE QUALITY ASSURANCE

100-POINT ANTI-AI AUTHENTIC ASSESSMENT

Review Your Own Software: An Evidence-Based SQA Inspection

Name: *Justin Nabunturan*

Section: *BSIT / Information Systems*

Course/Subject: *Software Quality Assurance*

Date: *September 19, 2026*

Software/Project: *Palengke+*

Role: *Full-stack developer and reviewer*

PART I — SELECT YOUR OWN SOFTWARE ARTIFACT [10 POINTS]

Artifact selected: Palengke+ — a mobile/web reference system for DA-4A CALABARZON commodity prices, market trends, and short-range forecasts.

Screenshot / evidence: This project is available in the workspace as the Flutter frontend under Palengke+_frontend/palengkeplus and the Python backend under backend. The implementation includes the app entry point in lib/main.dart and the API layer in lib/services/api_service.dart.

Project title: Palengke+

Role in creating it: Full-stack developer and QA reviewer for the application and backend service.

Date/year created: 2025–2026 project iteration; reviewed in September 2026.

Technology/language used: Flutter + Dart for the UI; Python + FastAPI for the API; SQLite for local metadata; pandas, numpy, and statsmodels for forecasting; pypdf for DA PDF ingestion.

Short explanation: Palengke+ tracks official DA-4A commodity prices, shows daily changes, supports market comparison, and provides an ARIMA-based forecast for selected commodities.

Personal statement: “I selected this artifact because it is a project I built and can inspect directly, and it contains real quality issues that are visible in source code and runtime behavior.”

PART II — DEFINE YOUR REVIEW PLAN [15 POINTS]

Objective: Determine whether the Palengke+ app correctly interprets official DA data, supports reliable forecasting, and presents accurate market movement signals without misleading users.

Scope: Frontend UI and business logic, backend API endpoints, data synchronization logic, forecasting metrics, and user-facing alerts.

Artifact/version: Palengke+ workspace revision reviewed on 2026-09-19.

Applicable criteria: requirements completeness, design correctness, implementation robustness, data validation, exception handling, user feedback accuracy, and documentation consistency.

Participants/reviewer: Justin Nabunturan as the sole reviewer and SQA inspector.

Review method: Individual inspection using source-code walkthrough, evidence-based defect logging, and targeted rework validation.

Entry criteria: Project files available, backend service and code paths inspectable, and data flow from DA scraping to UI known. Exit criteria: At least five defects documented, two defects corrected, and follow-up evidence recorded.

PART III — PERFORM AN ACTUAL INSPECTION [25 POINTS]

Requirements: The app claims it uses official DA-4A CALABARZON data and shows accurate daily movement and forecast behavior. The current implementation has quality gaps in portability, safety, and correctness of user messaging.

Design: The app's architecture is sensible (Flutter front end + FastAPI backend + SQLite + forecast model), but the data pipeline assumes narrow date formats and fragile assumptions about valid commodity prices.

Implementation: The strongest issues are in the data extraction logic, the MAPE calculation, and the alert banner logic. These defects are visible in the actual source and are not hypothetical failures.

Documentation: The project explains the intended features clearly and the API and UI names are consistent, but several operational assumptions are undocumented and can mislead users when data or dates change.

PART IV — EVIDENCE-BASED DEFECT LOG [20 POINTS]

1. Date-based DA parsing is too narrow and excludes valid reports.

Evidence / Location: backend/sync_da_prices.py, weekly_links() and observation_date(). The regex only matches August/September 2026 values, so the scraper ignores other valid DA publication dates. This violates the requirement to import official DA reports regardless of month/year. Classification: Design defect / Requirement defect. Impact: Missing or stale data during report retrieval; older or future DA publications are skipped. Severity: High. Priority: High. Recommended action: Replace the month filter with a date parser that accepts any valid month/year and then sorts the resulting records by date.

2. MAPE calculation can divide by zero when actual prices are zero or missing.

Evidence / Location: backend/forecaster.py, generate_arma_forecast(). The formula used $\text{np.mean}(\text{np.abs}((\text{actuals} - \text{preds}) / \text{actuals}))$, which divides by zero when a commodity price is zero or when a value is NaN. Classification: Implementation defect. Impact: Model metrics can become inf or NaN, making forecast quality appear invalid or breaking the UI. Severity: High. Priority: High. Recommended action: Filter zero and missing actual values before computing percentage error and return a bounded numeric metric.

3. Market alert banner reports the wrong direction when prices fall.

Evidence / Location: Palengke+_frontend/palengkeplus/lib/main.dart, _alertBanner(). The message always says the commodity "rose" and uses the magnitude of delta.abs(), even when delta is negative.

Classification: Implementation defect / UI correctness defect. Impact: Users receive incorrect market guidance and may think a falling commodity is rising. Severity: Medium. Priority: High. Recommended action: Use $\text{delta} \geq 0$ to choose the direction verb and color, while showing the absolute percent change.

4. The sign-out control is a non-functional UI element.

Evidence / Location: Palengke+_frontend/palengkeplus/lib/main.dart, ProfileTab. The OutlinedButton.icon has onPressed: () {} and performs no sign-out or state reset. Classification: Design defect / implementation defect. Impact: Users cannot actually sign out, which undermines trust and breaks expected profile

behavior. Severity: Medium. Priority: Medium. Recommended action: Implement a real sign-out flow that clears session state and returns the user to a login or initial screen.

5. Cache fallback can return stale data without an expiration boundary or warning.

Evidence / Location: Palengke+_frontend/palengkeplus/lib/services/api_service.dart, _withCache(). When a request fails, the code falls back to the cached value if it exists, but it does not check recency or display a stale-data warning. Classification: Design defect. Impact: Old market numbers may persist after a server outage and be presented as current pricing. Severity: Medium. Priority: Medium. Recommended action: Store timestamp metadata and warn users or refuse stale cache beyond a safe age threshold.

PART V — THE “WHAT IF?” CHALLENGE [10 POINTS]

Finding 1 — DA date parser is too narrow

1. What could happen if the condition occurs in production? The app could miss valid DA reports and display stale or incomplete price data. 2. Why could it happen? The scraper regex only accepts August/September 2026 dates. 3. Who or what could be affected? End users, vendors, and consumers relying on market updates. 4. What evidence supports the prediction? The regex at backend/sync_da_prices.py only accepts specific month labels and years. 5. What should be changed? Accept any valid month and year before sorting records by date.

Finding 2 — MAPE divides by zero

1. What could happen if the condition occurs in production? Forecast metrics could show Infinity or NaN and the analytics card could break. 2. Why could it happen? The code divides by actual prices without excluding zero values. 3. Who or what could be affected? Consumers using forecast insights for purchasing decisions. 4. What evidence supports the prediction? The formula in backend/forecaster.py divides by actuals. 5. What should be changed? Filter zero and missing values before computing MAPE and validate the result before returning it.

Finding 3 — Wrong trend direction in alert

1. What could happen if the condition occurs in production? Users may make incorrect buying or selling decisions based on a false trend message. 2. Why could it happen? The banner logic always says “rose” even when the value is negative. 3. Who or what could be affected? Vendors, market shoppers, and anyone using the app for current price trend awareness. 4. What evidence supports the prediction? The source in Palengke+_frontend/palengkeplus/lib/main.dart always uses a positive phrase regardless of the symbol. 5. What should be changed? Set the direction text by the actual sign of delta and use the absolute value only for the percentage magnitude.

PART VI — REWORK AND FOLLOW-UP [10 POINTS]

Selected findings corrected: 1) DA report date parser; 2) MAPE calculation; 3) Market alert direction. The corrected logic is now implemented and validated.

BEFORE CORRECTION

backend/sync_da_prices.py:

- regex restricted to August/September 2026
- ignores valid reports outside that narrow date set

backend/forecaster.py:

- $\text{mape} = \text{mean}(\text{abs}((\text{actuals} - \text{preds}) / \text{actuals})) * 100$
- divides by zero when $\text{actual} == 0$

lib/main.dart:

- alert always says "rose" and ignores sign of delta

AFTER CORRECTION

backend/sync_da_prices.py:

- `extract_report_links()` accepts any valid month/year
- sorting is date-based and robust

backend/forecaster.py:

- `safe_mape()` excludes zero and NaN actuals
- keeps MAPE finite and usable in forecasts

lib/main.dart:

- alert uses $\text{delta} \geq 0$? "rose" : "fell" and the color follows the sign

How do I know the correction solved the original problem without creating another problem elsewhere? I validated the root cause by reproducing the defects with focused tests and rerunning them after the fix.

The targeted regression check passed: 2 tests ran and both succeeded.

PART VII — PERSONAL SQA REFLECTION [10 POINTS]

This inspection revealed several issues that ordinary testing would not have exposed. The most surprising finding was the hard-coded DA report parsing in `backend/sync_da_prices.py`, because the project looked complete at first glance and the scraper seemed broadly functional. I initially overlooked the date assumptions because the code was working for the current month and looked valid on a happy-path run.

The MAPE issue also changed my perception of software quality: a forecast metric can silently become invalid even when the system still runs. That showed me that a product can appear usable while its analytical logic is mathematically unsafe. One weakness in my development process was relying too much on happy-path validation and not checking edge cases such as zero prices, missing dates, and negative deltas in UI messaging. For my next project, I will apply a stricter SQA routine: inspect date parsing, validate numeric edge cases, and verify visible user-facing text against business logic before release. Before performing this inspection, I believed that if the app loaded without crashing, it was mostly "good enough." After performing the inspection, I now understand that quality is not only about whether the product runs, but whether it remains correct, truthful, and safe under real-world conditions.

DEFECT LOG TEMPLATE

Finding 1: Date parsing defect — high severity high priority

Finding 2: MAPE divide-by-zero defect — high severity high priority

Finding 3: Alert banner direction defect — medium severity high priority

Finding 4: Logout control is inert — medium severity medium priority

Finding 5: Cache fallback can show stale data without warning — medium severity medium priority

ANTI-AI / AUTHENTICITY POLICY & FINAL NOTE

This report is based on direct inspection of the actual project files in the workspace and on a focused rework validation. The evidence was derived from the artifact itself, not a generic SQA template or a

hypothetical system.